Cahier des Charges

Projet : Dashboard de Surveillance Météo & Qualité de l'Air

Niveau : 3e Année d'École d'Ingénieur (FISE / FISA) | **Format :** Groupes de 3 étudiants
Volume Horaire : ~ 10 heures / étudiant (30 heures-homme / équipe) | **Livrable :** Dépôt Git

1. Contextualisation et Objectif du Projet

Dans le cadre des enjeux liés à la transition écologique et à la santé publique, le suivi en temps réel des conditions météorologiques et de la qualité de l'air est devenu un sujet crucial.

L'objectif de ce projet est de concevoir et développer une **application Python complète** permettant de :

1. **Récupérer automatiquement** des données météo et environnementales en temps réel et prévisionnelles via des APIs REST publiques.
2. **Traiter, nettoyer, enrichir et stocker** ces données de manière optimisée.
3. **Restituer ces informations** au travers d'un **dashboard interactif** intégrant des visualisations graphiques avancées, des indicateurs de performance (KPI) et un système d'alerte automatisé.

2. Exigences Techniques et Libellé des Besoins

2.1. Fonctionnalités Obligatoires (MVP – Minimum Viable Product)

A. Ingestion et Stockage de Données

- **Multi-sources & Multi-villes :** Récupération automatique de données pour au moins **5 villes au choix** (ex: Paris, Lyon, Marseille, Londres, Madrid).
- **Données météo à collecter :** Température actuelle, ressentie, min/max, humidité, pression atmosphérique, vitesse/direction du vent, couverture nuageuse, précipitations.
- **Données de qualité de l'air à collecter :** Indice de qualité de l'air (AQI), concentrations des polluants ($PM_{2.5}$, PM_{10}, NO_2, O_3, CO, SO_2).
- **Mise en cache & Persistance :** Implémenter un système de cache local (base SQLite ou fichiers JSON) avec un délai d'expiration (TTL, ex: 30 minutes) pour limiter les appels API superflus.

B. Traitement et Analyse Métier

- **Statistiques & Agrégations :** Calcul de l'indice moyen/médian sur les N derniers jours/heures, tendances d'évolution à 24h et 7 jours.

- **Indice Global de Santé/Confort :** Calcul d'un score composite personnalisé combinant température ressentie, humidité et niveaux de pollution.
- **Système d'Alertes :** Détection automatique des dépassements de seuils réglementaires (normes OMS) et génération de notifications visuelles dans le dashboard (*"Alerte Canicule"*, *"Alerte Pic de Pollution NO_2"*).

C. Interface Utilisateur & Visualisation

- **Interface Web Interactive :** Développée avec **Streamlit** (ou alternativement Dash / Gradio).
- **Composants d'IHM attendus :** Panneau de sélection (ville, dates, métriques), cartes KPI synthétiques, graphiques temporels interactifs, et une carte géographique interactive (*folium* / *plotly*) avec un code couleur selon l'indice AQI.

2.2. Fonctionnalités Avancées / Bonus (Au choix)

- **Prédictions par Machine Learning simple :** Régression linéaire ou modèle autorégressif (*scikit-learn*) pour prédire la température ou le niveau de $PM_{2.5}$ à $H + 24$.
- **Exportation de rapports :** Génération automatique de synthèses au format PDF ou CSV à destination d'un utilisateur final.
- **Conteneurisation :** Livrer l'application sous forme de conteneur **Docker** (`Dockerfile` + `docker-compose.yml`).
- **Mode Déconnecté / Simulation :** Mode fallback qui injecte un jeu de données simulant les réponses API en format JSON en cas de coupure réseau.

3. APIs Recommandées (Accès Gratuit Sans Carte Bleue)

Les groupes sont libres d'utiliser les APIs REST de leur choix :

1. **Open-Meteo API** (`https://open-meteo.com/`) : Totalement gratuite, **aucune clé requise**, données météo et qualité de l'air complètes.
2. **OpenWeatherMap** (`https://openweathermap.org/api`) : Clé API gratuite requise (limite 60 req/min).
3. **WAQI (World Air Quality Index)** (`https://aqicn.org/api/`) : Stations de mesure réelles à travers le monde.

4. Architecture Technique & Bonnes Pratiques

4.1. Structure de Projet Recommandée

Arborescence standard du projet

```
dashboard_meteo/

   data/                    # Cache local, BD SQLite ou données mock
   src/
       __init__.py
       api_client.py        # Requêtes API et gestion du cache
       data_processing.py   # Nettoyage, calculs Pandas, statistiques
       alerts.py            # Moteur de règles et d'alertes
       dashboard.py         # Application UI (Streamlit / Dash)

   tests/                   # Tests unitaires (pytest)
       test_api.py
       test_processing.py

   .gitignore
   requirements.txt         # Dépendances du projet
   README.md                # Documentation complète
```

4.2. Exigences Qualité Logicielle

- **Modularité & POO :** Séparation stricte de la récupération des données, du traitement métier et de la couche d'affichage (Pattern type MVC).
- **Gestion des exceptions :** Traitement explicite des erreurs réseau (`requests.exceptions.RequestExcep` clés invalides et JSON corrompus.
- **Typage et Docstrings :** Utilisation des *type hints* (ex: `def get_weather(city: str) -> dict:`) et docstrings.
- **Variables d'environnement :** Stockage des clés d'API dans un fichier `.env` (exclu du commit Git).

5. Organisation du Travail & Répartition Suggérée

Étudiant A : Ingénieur Données & API Integration
Création du module `api_client.py`. Implémentation des requêtes HTTP, géocodage, gestion du cache local (SQLite/JSON) avec TTL, et gestion des erreurs.

Étudiant B : Data Analyst & Logique Métier
Implémentation de `data_processing.py` et `alerts.py`. Structuration Pandas, nettoyage des données, calcul des tendances, définition des règles d'alerte et rédaction des tests `pytest`.

Étudiant C : Lead UI / UX & Visualisation
Développement du dashboard `dashboard.py`. Implémentation des filtres, affichage des cartes KPI, intégration des graphiques Plotly et de la carte interactive.

6. Planning & Grille d'Évaluation

6.1. Planning Prévisionnel (Séquencement sur 10h)

Phase	Durée	Objectifs & Activités
Phase 1	1,5 h	Cadrage, initialisation Git, étude des documentations d'API, architecture.
Phase 2	3,5 h	Dédev. client API avec cache (A), scripts Pandas (B), maquette UI (C).
Phase 3	3,0 h	Intégration des modules dans le dashboard, graphiques Plotly, moteur d'alertes.
Phase 4	2,0 h	Tests unitaires, données mockées, nettoyage du code, rédaction du README.

6.2. Barème d'Évaluation Indicatif

Critères	Pondération	Éléments Évalués
Architecture & Qualité du Code	**30%**	Modularité, respect POO, typage, docstrings, gestion propre des exceptions.
Fonctionnalités Obligatoires (MVP)	**30%**	Appels API, cache fonctionnel, calculs statistiques exacts, moteur d'alertes.
Ergonomie & Visualisation (UI/UX)	**20%**	Lisibilité du dashboard, pertinence des graphiques, carte interactive.
Pratiques DevOps & Tests	**10%**	Commits Git réguliers, présence et passage des tests `pytest`.
Documentation (README)	**10%**	Clarté du guide de lancement, explications techniques et exemples.